
Software Qualities: Memory Efficiency

Marco Faella

Why space efficiency?

Because you might have **little space** or **a lot of stuff**

Little space:

- Embedded systems
- Modern cars have 50-100 microcontrollers on board
- Most microcontrollers come with less than 1MB RAM

A lot of stuff:

- Recent AAA videogames include over 50GB of data
- Big data applications
- Billions of water containers!

Object Layout

Memory overheads

Java objects include an *object header* with auxiliary information

The header supports the following functionalities:

1. Type awareness and reflection
2. Multi-threading
3. Garbage collection
4. Identity hashcode

The size of a reference

In theory:

References (aka pointers) take 32 or 64 bits depending on the architecture

In practice:

Enters *Compressed ordinary object pointers* (COOPs):

- Most programs don't address more than 32GB
- Encode references using 32 bits
- When using an address, **add 3 zeros** at the end
- You get 35 bit addresses: 32GB of addressable memory

ON by default in *HotSpot*

It can be turned off with cmdline arguments to the JVM

1. Type awareness overhead

Each object must know its *dynamic type*

This is used by `instanceof`, `getClass` and other reflective operations

The header contains **a pointer to a Class object**

2. Multi-threading overhead

In theory:

Java assigns a **monitor** to each object

- Similar to a mutex
- Accessed via the `synchronized` keyword

In practice:

Current versions of HotSpot instantiate the monitor *on demand*

This reduces (but does not avoid) the memory overhead

3. Garbage collection overhead

In theory:

A reference count for each object

In practice:

- HotSpot employs *tracing* (mark-and-sweep) and *generational* GCs
- Tracing:
 - No reference count
 - Instead, follow all references starting from GC *roots*

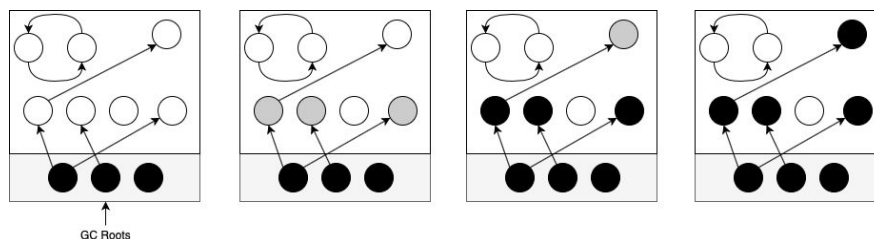


Image by
[Ali Dehghani](#)

- Generational:
 - Objects are arranged in groups called generations, based on the time since creation
 - **Objects carry an *age* field**

4. Identity hashCode overhead

1. The value returned by `Object.hashCode()` must be **constant** during the lifetime of an object
2. However, some garbage collectors can *move* (relocate) objects around!
See compacting garbage collectors

So: the original hashCode provided by Object must be **stored**

This is called the *identity hashCode*

Other overheads: alignment and padding

In most architectures, memory accesses are more efficient if they are ***word-aligned***

In a 64-bit arch, an address is word-aligned if it is a **multiple of 8**

- lower 3 bits equal to zero
- just like COOPs!

Effective object sizes are multiples of 8 bytes

If necessary, empty space is inserted into objects (**padding**)

Padding in C structs

Padding happens also in C: a struct may take *more space than strictly necessary*

Try this in Compiler Explorer:

```
#include <stdio.h>

struct msg {
    int n;
    char s[5];
};

int main(void) {
    char s[] = "ciao";
    printf("s: %d\n", sizeof(s));
    printf("struct: %d\n\n", sizeof(struct msg));

    struct msg x = { 42, "ciao" };
    for (int i=0; i<sizeof(struct msg); i++)
        printf("%d\t%c\n", (int) ((char*)&x)[i], ((char*)&x)[i]);
}
```

Summary of overheads

64bit arch with COOPs:

- Pointer to Class object 4 bytes
- Monitor + GC + hashCode 8 bytes (complex dynamic encoding)

Total: **12 bytes**

For simplicity, ignore alignment and padding

For details, see HotSpot source

- File src/share/vm/oops/markOop.hpp
- at <https://hg.openjdk.java.net/jdk10/jdk10/hotspot>

Array overheads

Arrays store:

- **Their size**
 - **4 additional bytes**
- The static type of their cells
 - No additional overhead
 - This information is part of their type
 - A `String[]` contains a reference to the Class object for `String[]`

Total: **16 byte** overhead

Reference implementation

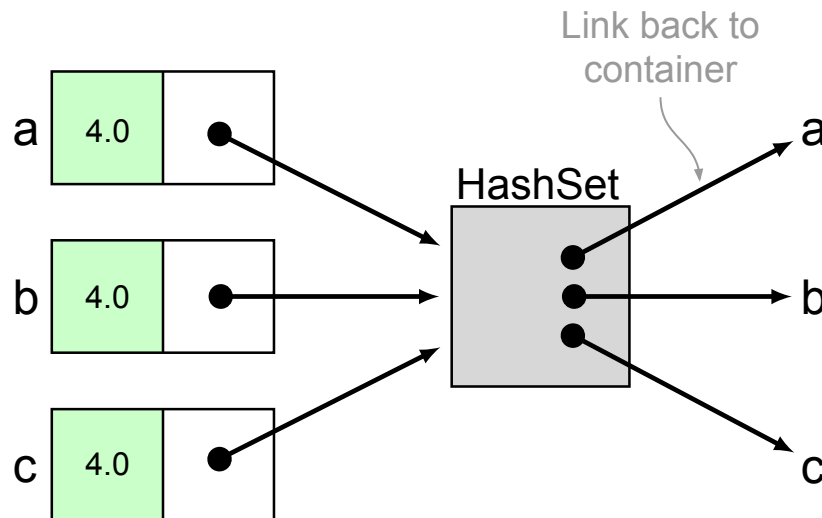
The **fields**:

`double amount`

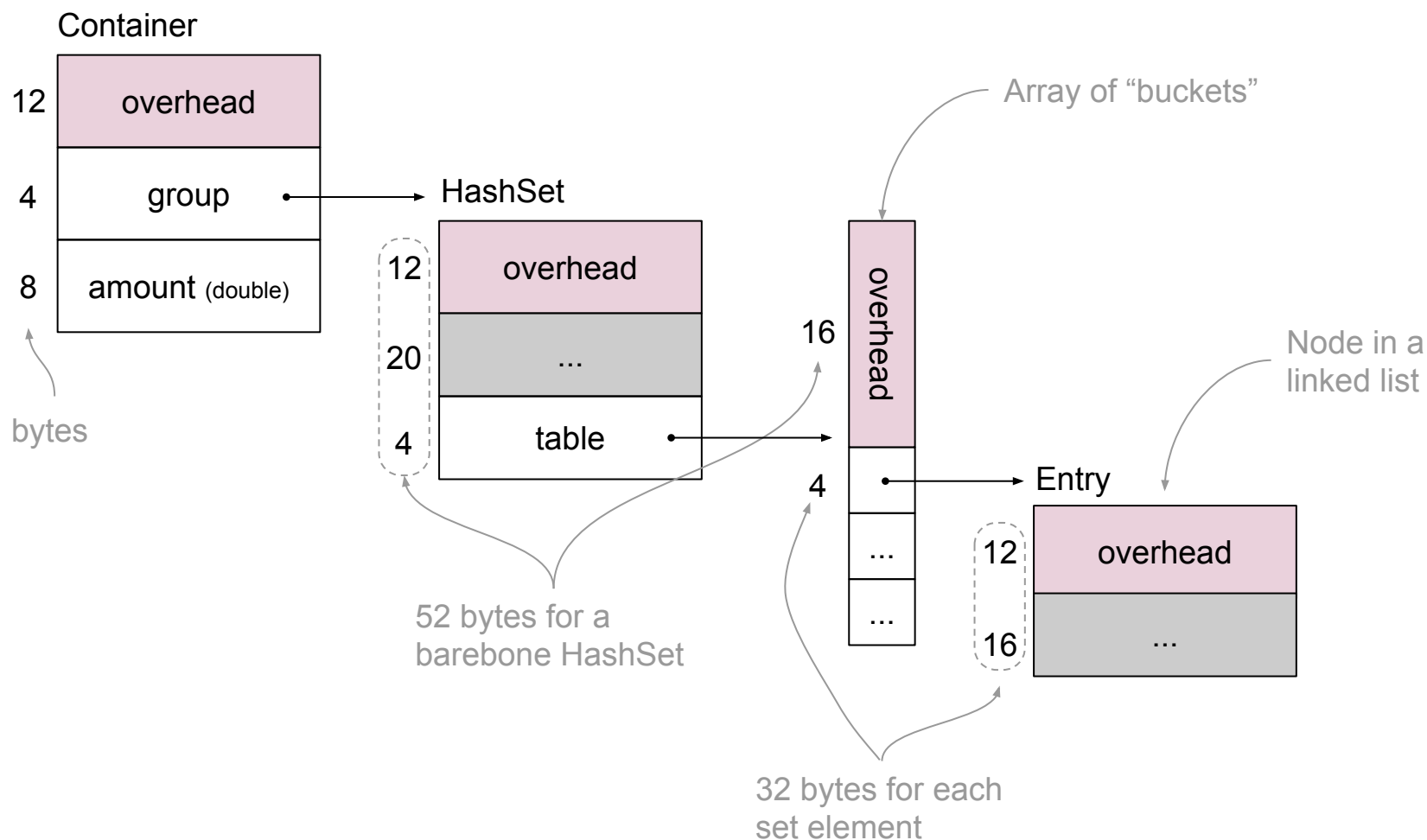
Amount of water in this container

`Set<Container> group`

Containers connected *directly or indirectly* to this one, including this one



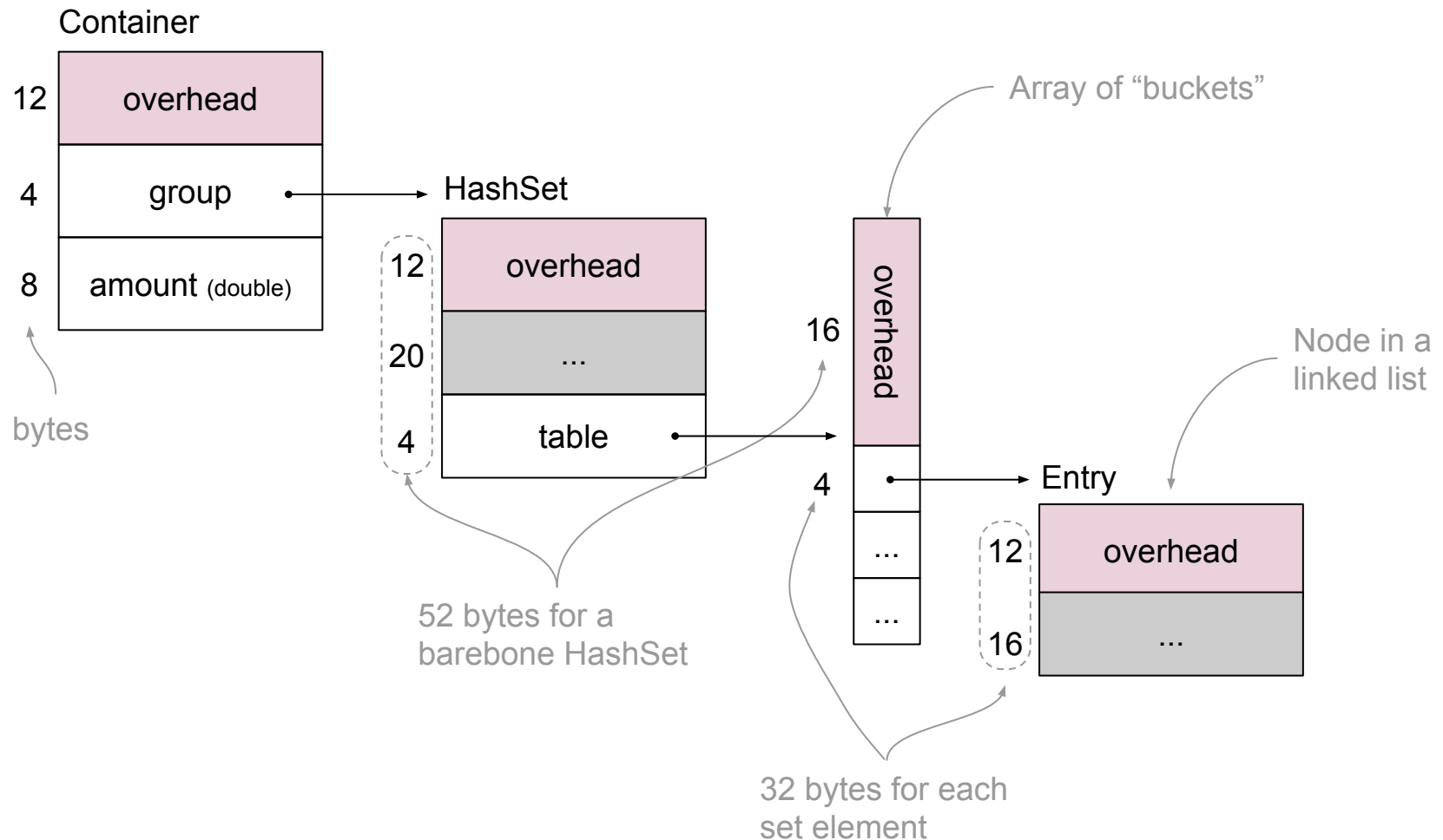
Detailed memory layout of Reference



Con

Table 2.1 Memory requirements of Reference in two conventional scenarios

Scenario	Size (calculations)	Size (bytes)
1000 isolated	$1000 * (12 + 8 + 4 + 52 + 32)$	108000
100 groups of 10	$1000 * (12 + 8 + 4) + 100 * (52 + 10 * 32)$	61200

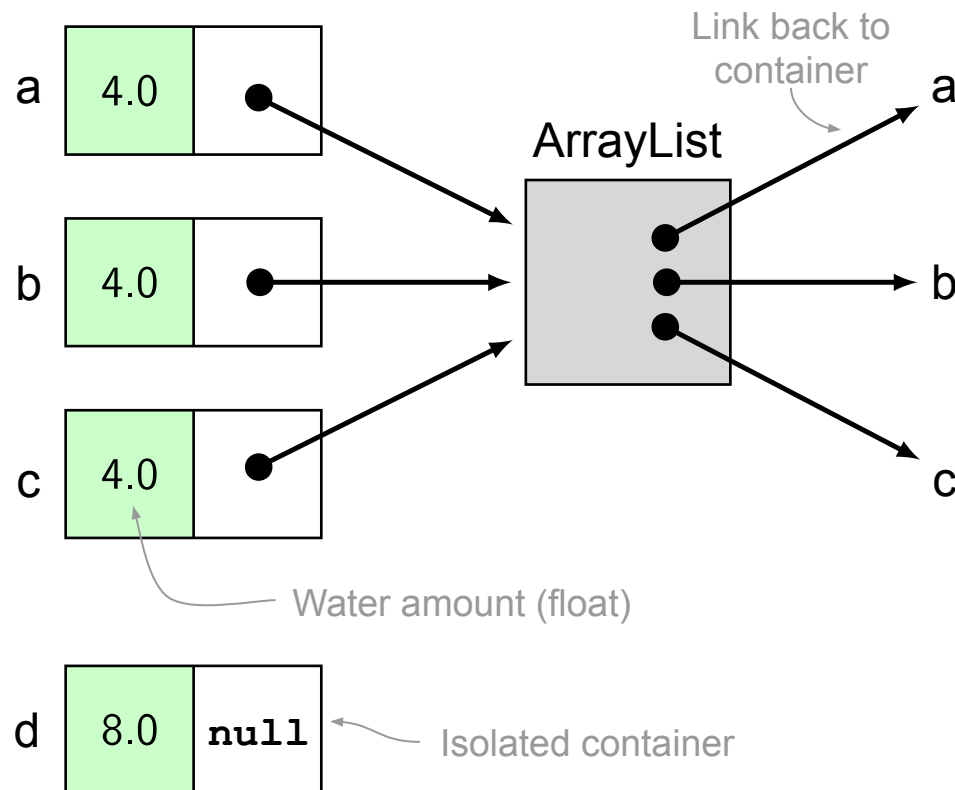


Memory Efficiency

Gently squeezing

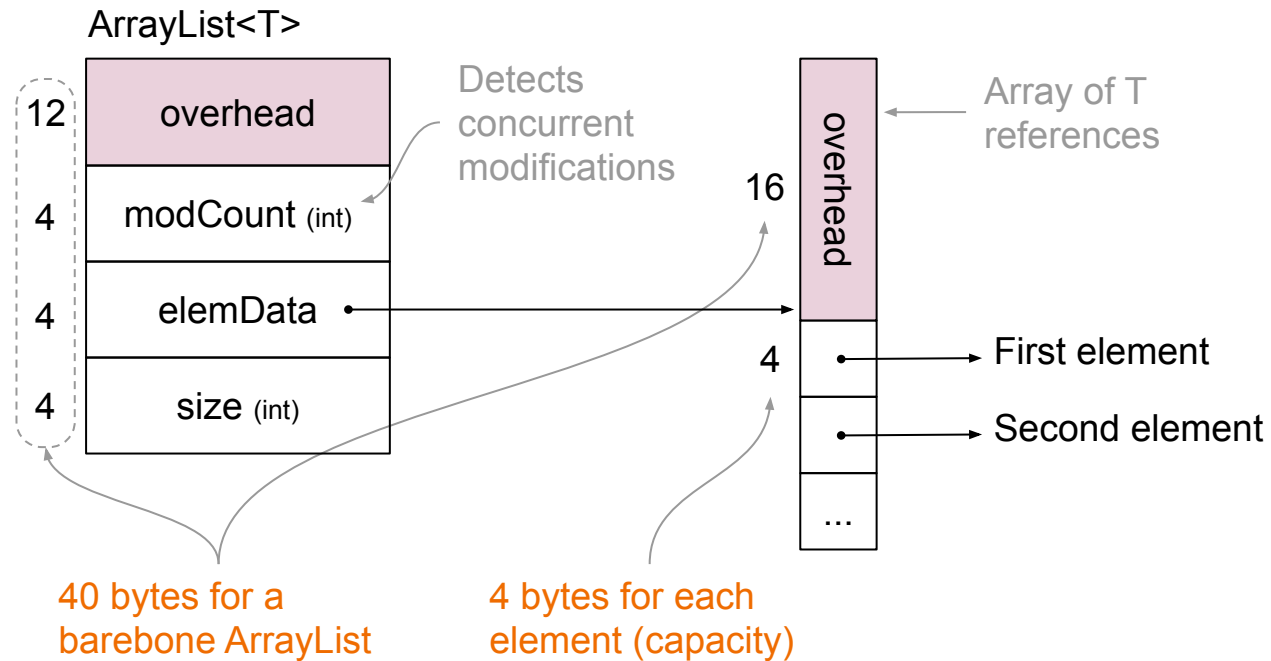
[Memory1]

1. Amount field: Switch from double to float
2. Group field: Switch from HashSet to a *lazily instantiated* ArrayList



Note: methods keep the same complexity as Reference

Memory requirements of ArrayList



Memory requirements of Memory1

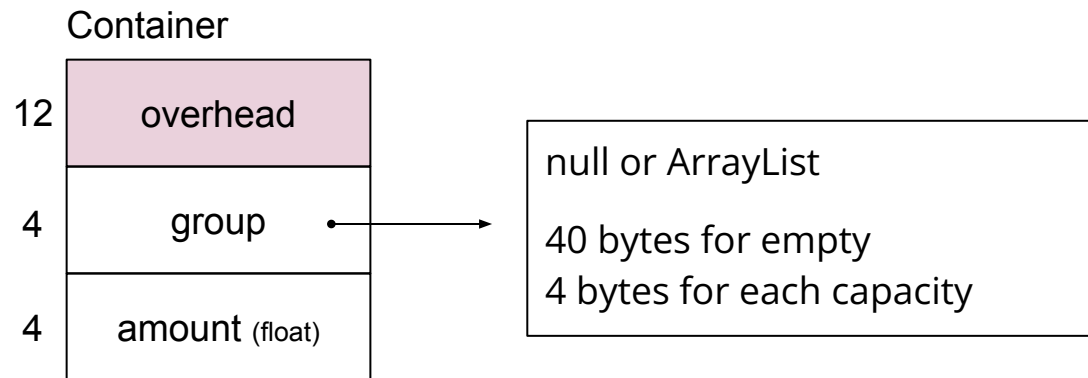


Table 4.1 Memory requirements of *Memory1*

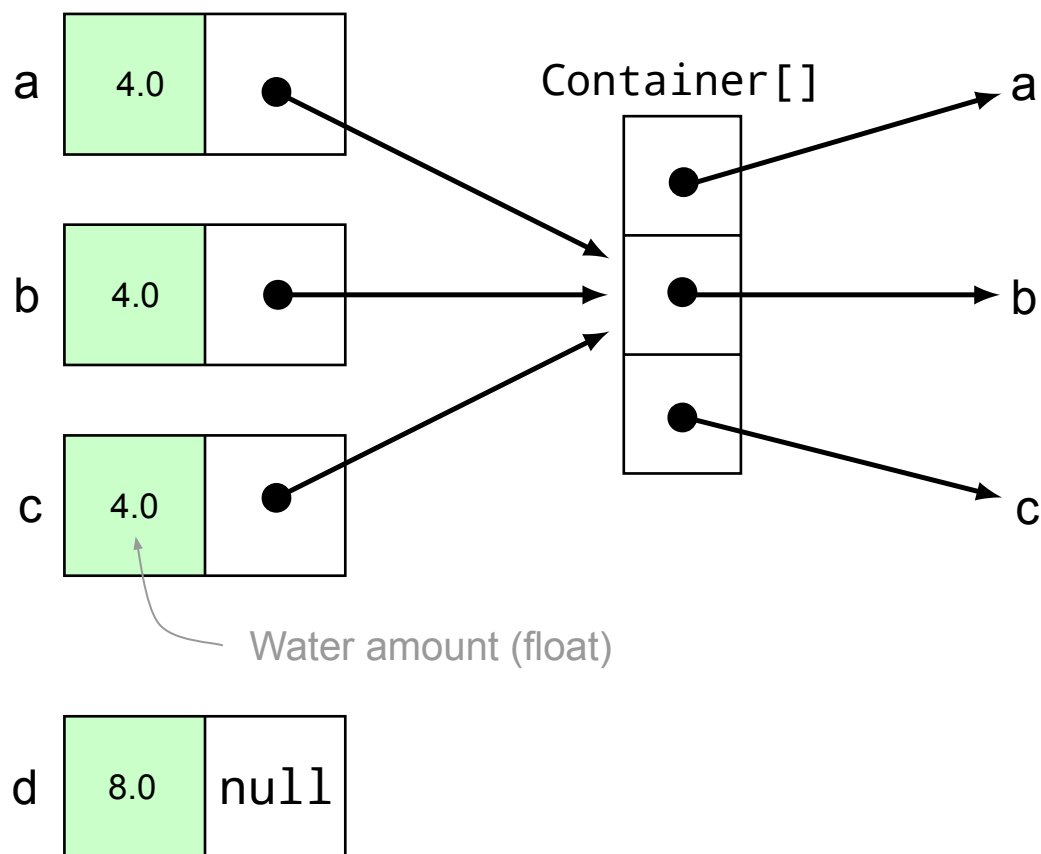
Scenario	Size (calculations)	Size (bytes)	% of reference
1000 isolated	$1000 * (12 + 4 + 4)$	20000	19%
100 groups of 10	$1000 * (12 + 4 + 4) + 100 * (40 + 10 * 1.25 * 4)$	29000	47%

Expected excess capacity

Plain arrays

[Memory2]

Represent group as an **array of Containers**



Memory requirements of Memory2

Table 4.4 Memory requirements of common collections, assuming that the capacity of the `ArrayList` and the `HashSet` is equal to their size. The second column is tagged “barebone” instead of “empty” because it doesn’t take into account the default initial capacity of that collection.

Type	Size (barebone)	Size (each extra element)
array	16	4
<code>ArrayList</code>	40	4
<code>LinkedList</code>	24	24
<code>HashSet</code>	52	32

Table 4.3 Memory requirements of *Memory2*

Scenario	Size (calculations)	Size (bytes)	% of reference
1000 isolated	$1000 * (12 + 4 + 4)$	20000	19%
100 groups of 10	$1000 * (12 + 4 + 4) + 100 * (16 + 10 * 4)$	25600	42%



A modest improvement over Memory1!

An object-less API

Containers identified by **integers** instead of objects

```
int a = Container.newContainer(),  
    b = Container.newContainer(),  
    c = Container.newContainer(),  
    d = Container.newContainer();
```

Replaces the constructor

```
Container.addWater(a, 12);  
Container.addWater(d, 8);  
Container.connect(a, b);  
System.out.println(Container.getAmount(a));
```

Constructor and methods become **4 static methods**

Not object-oriented!

Two IDs and two arrays

[Memory3]

- Each container is identified by an ID (**containerID**)
- Each group of containers is identified by an ID (**groupID**)

Class sketch:

```
public class Container {  
    // From containerID to groupID  
    private static int group[] = new int[0];  
    // From groupID to the amount in each container in the group  
    private static float amount[] = new float[0];  
  
    public static float getAmount(int containerID) {  
        int groupID = group[containerID];  
        return amount[groupID];  
    }  
    ...  
}
```

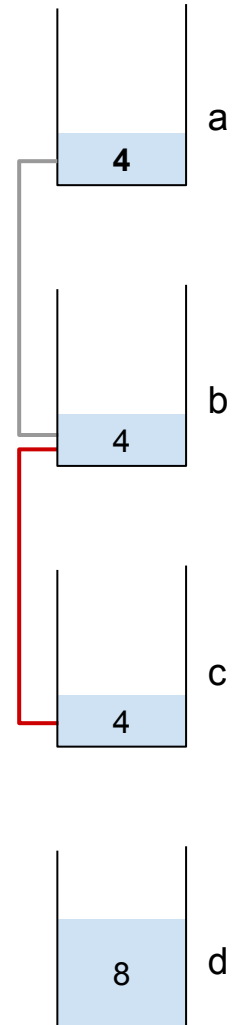

Memory layout of Memory3

	group
0 (a)	0
1 (b)	0
2 (c)	0
3 (d)	1

Every container
in group 0 holds
4 units of water.

Container
2 belongs
to group 0.

	amount
0	4.0
1	8.0



Method implementation

Method **newContainer**:

- Adds a new containerID and a new groupID
- Enlarge both arrays
- **Linear** time (*)

Method **addWater**:

- Counts the size of the group (scans the group array)
- Updates the amount array
- **Linear** time (*)

Method **connect**:

- Merges two groups
- Removes one groupID
- To discover all containers in a group, you have to iterate over **all containers**
- **Linear** time (*)

All methods except getAmount are linear (*)

(*) in the total number of containers

Memory requirements of Memory3

Table 4.6 Memory requirements of *Memory3*

Scenario	Size (calculations)	Size (bytes)	% of reference
1000 isolated	$4 + 16 + 1000 * 4 + 4 + 16 + 1000 * 4$	8040	7%
100 groups of 10	$4 + 16 + 1000 * 4 + 4 + 16 + 100 * 4$	4440	7%

The black hole

[Memory4]

- Each group is represented by a **linked list** of containers
- Only the *last* container in each group records the water amount
- A **single array** of float stores both the *next pointer* and the *amounts*
 - Similar to a C union

```
public class Container {  
    private static float nextOrAmount[] = new float[0];
```

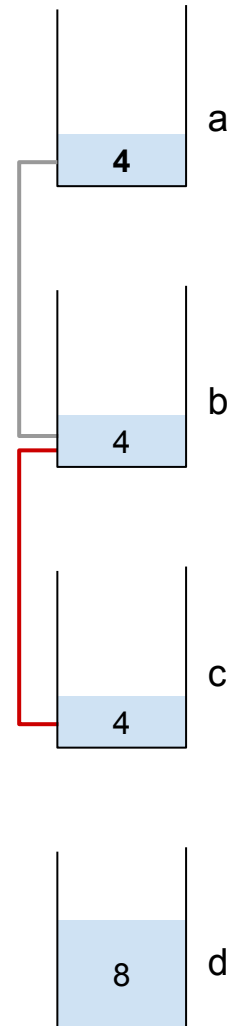
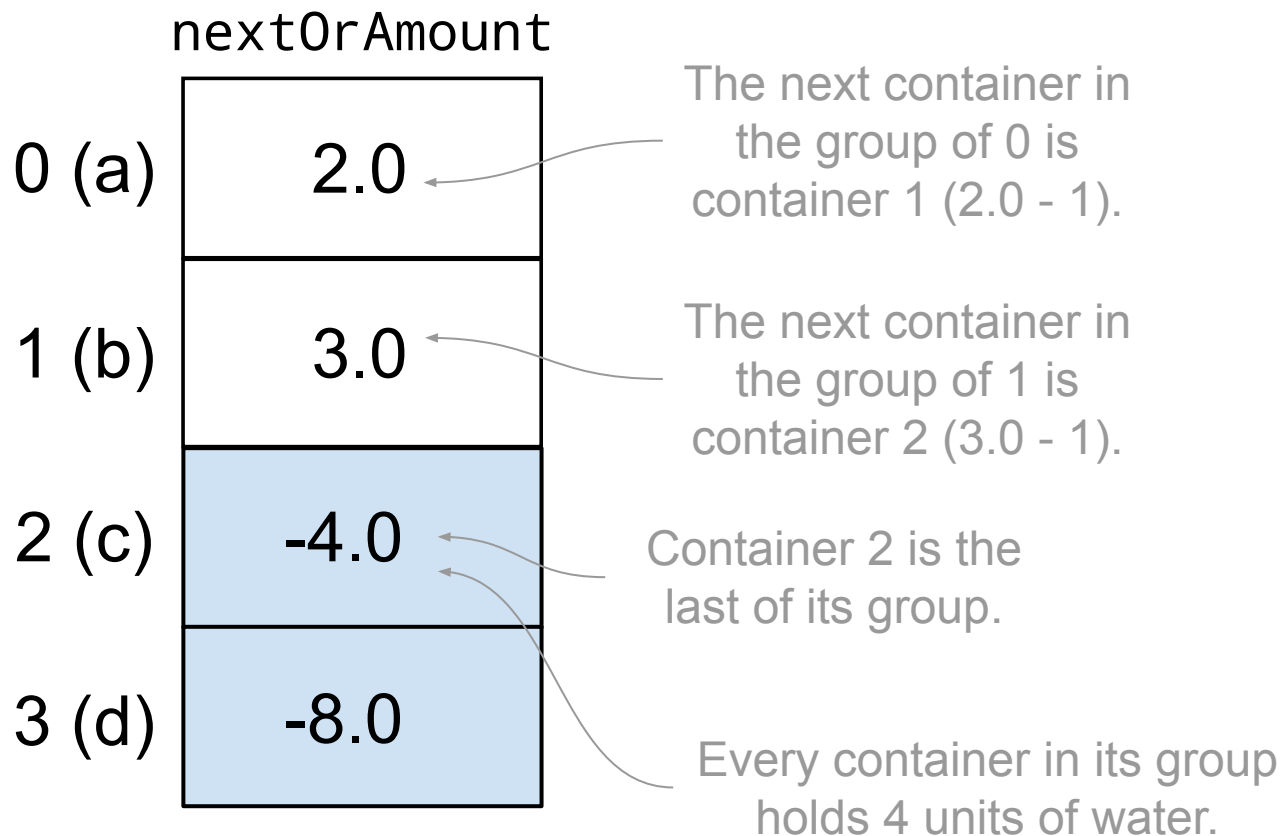
When nextOrAmount[i] is **positive**:

it's the index of the **next** container in this group, with a +1 bias

When it's **negative** or **zero**:

it's the *opposite* of the water **amount** in each container of this group

Memory layout of Memory4



Implementation of getAmount

[Memory4]

```
public class Container {  
    private static float nextOrAmount[] = new float[0];  
  
    public static float getAmount(int containerID) {  
        while (nextOrAmount[containerID]>0)  
            containerID = (int)nextOrAmount[containerID] -1;  
        return -nextOrAmount[containerID];  
    }  
    ...  
}
```

Weakness of Memory4

- It works as long as the index of the next container can be represented by a float
- How many containers are supported?
- Recall that a float is 32 bits (24 bits significand + 8 bit exponent)
- The *uninterrupted integer range* of the type float is 0 to 2^{24}

```
> jshell
```

```
> float f = 1 << 24;  
1.6777216E7
```

```
> f+1 == f;  
true
```

```
> f-1 == f;  
false
```

Method implementation

[Memory4]

Method **addWater**:

- Find the last container in the group
- Count the size of the group
- **Quadratic** time (in the total number of containers)

Method **connect**:

- Find all containers in both groups
- **Quadratic** time (in the total number of containers)

Table 4.9 Time complexities of *Memory4*

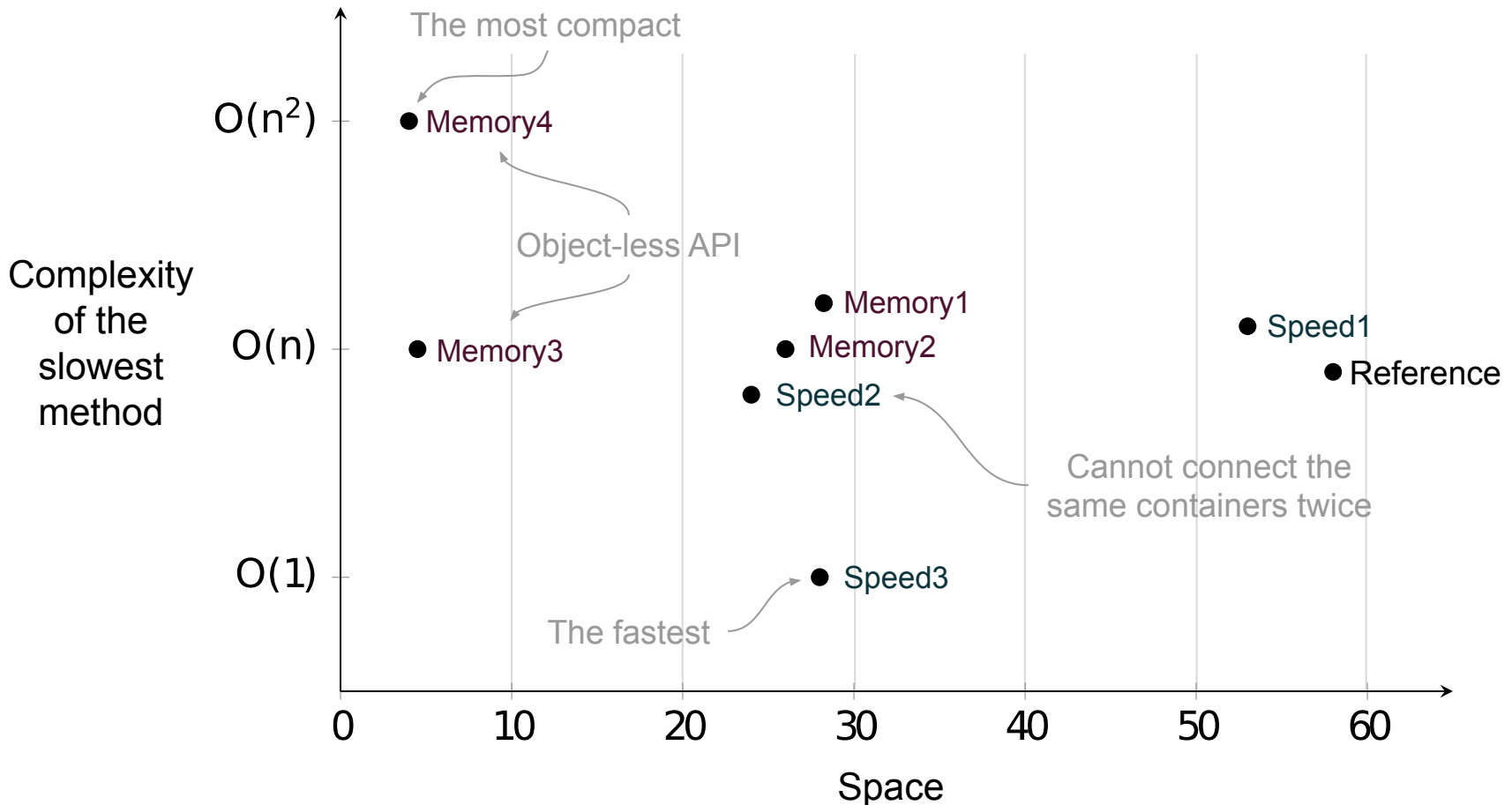
Method	Time complexity
getAmount	$O(n)$
connectTo	$O(n^2)$
addWater	$O(n^2)$

Summary of memory requirements

Table 4.10 Memory requirements of all implementations from this chapter, plus *Reference*. Recall that *Memory3* and *Memory4* expose a different, object-less API.

Scenario	Version	Bytes	% of <i>Reference</i>
1000 isolated	<i>Reference</i>	108000	100%
	<i>Memory1</i>	20000	19%
	<i>Memory2</i>	20000	19%
	<i>Memory3</i>	8040	7%
	<i>Memory4</i>	4020	4%
100 groups of 10	<i>Reference</i>	61200	100%
	<i>Memory1</i>	29000	47%
	<i>Memory2</i>	25600	42%
	<i>Memory3</i>	4440	7%
	<i>Memory4</i>	4020	7%

Comparing implementations



(Bytes per container when 1000 containers are connected in 100 groups of 10)

Exercise: analyze SparseArray

The Android class `android.util.SparseArray` is a memory-efficient implementation of an array of objects, whose indices can be arbitrary integers instead of a contiguous interval starting at 0. As such, it's a replacement for `Map<Integer,Object>`.

Internally, it uses two arrays: one for the indices (aka the keys) and one for the objects (aka the values).

Check out the source code for the `SparseArray` class (<http://mng.bz/DNZA>) and answer the following questions:

1. How much memory does an empty `SparseArray` created with `new SparseArray()` need?
2. How much memory does a `SparseArray` need that contains 100 objects with contiguous indices 0–99 (not counting the memory occupied by the objects themselves)?
3. How much memory does a `SparseArray` need that contains 100 objects with random integer indices?